sa

SAIMS

5-Day Execution Plan

Smart Assignment & Internship Management System

From zero to staging in 5 working days

Version 1.0 · March 26, 2026

5-Day Overview

Each day has a single focus theme. Teams work in parallel but sync twice daily (start-of-day kickoff + end-of-day check-in). The backend team shares a Postman collection at the end of each day — frontend must not wire real APIs until that collection is shared.

Day	Date	Theme	Focus	Key Risk
Day 1	Monday	Project setup & infrastructure	Setup sprint	Docker/compose issues block entire team
Day 2	Tuesday	Authentication & dashboards	Auth first	Auth bugs affect every feature built after
Day 3	Wednesday	Assignments & file submissions	Core loop	S3/Cloudinary credentials not ready
Day 4	Thursday	Grading, internship & notifications	Feedback loop	Grade→notify flow not tested E2E
Day 5	Friday	Admin panel, reports & deployment	Ship it	No staging deployment = no demo

> **Golden rule:** Backend team shares Postman collection at EOD every day. Frontend uses MSW mocks until the collection arrives — never block on each other.

DAY 1 — Project Setup & Infrastructure

Monday · Setup sprint

Everyone starts together. No feature code ships until the foundation is solid.

All Teams

- Create GitHub repo — set branch protection rules (main, develop, feature/*)
- Create project board (GitHub Projects or Trello) with columns: To Do / In Progress / Review / Done
- Agree on folder structure (src/routes, controllers, services, middleware, models) and commit message format
- Set up shared .env.example file with all required keys documented

Backend Team

- Scaffold Node.js + Express app with folder structure: src/routes, controllers, services, middleware, models
- Connect PostgreSQL with pg-pool; confirm connection with a health check route GET /health
- Write and run all 10 DB migration scripts: users, departments, courses, course_enrollments, assignments, submissions, grades, internships, internship_logs, notifications, audit_logs
- Add seed script: 1 admin, 2 teachers, 5 students — all with bcrypt hashed passwords
- Set up global error handling middleware returning { success: false, message, errors } envelope

Frontend Team

- Create React app with Vite + Tailwind CSS; confirm dev server runs
- Set up React Router v6 with placeholder pages for all 10 routes (empty shells with page title only)
- Create ProtectedRoute wrapper component that checks Zustand auth store and redirects to /login
- Set up Axios instance with base URL from .env; add JWT interceptor and 401 redirect logic
- Set up Zustand auth store: token, user object, role, setAuth(), clearAuth()

Testing Team

- Install and configure Jest + Supertest in the backend project; confirm npm test runs with a dummy passing test
- Install and configure Playwright in an /e2e folder; confirm npx playwright test executes without errors
- Write the master test plan document: list all test case IDs TC-001 to TC-012 with description, type (unit/integration/E2E), and expected result
- Validate seed data is correct: query the local DB and confirm 1 admin, 2 teachers, 5 students exist with correct roles and hashed passwords
- Write a smoke test for GET /health endpoint: assert 200 response and DB connection alive
- Set up GitHub Issues bug report template: Title / Environment / Steps to Reproduce / Expected Behaviour / Actual Behaviour / Severity / Screenshot

DevOps Team

- Write docker-compose.yml with 4 services: api (port 3001), frontend (port 3000), postgres (port 5432), redis (port 6379)
- Confirm every team member can run docker-compose up successfully — fix blockers before Day 2
- Set up GitHub Actions CI workflow: trigger on PR to develop → run npm test + npm run lint → block merge on failure
- Add .env to .gitignore; document all required env variables in README

End-of-day deliverables

Deliverable	Acceptance criteria
docker-compose up	All 4 containers running with no errors
DB migrations	All 10 tables exist; seed data queryable
React routing	10 placeholder pages accessible via URL
CI pipeline	GitHub Actions runs green on first PR
Jest + Playwright	Both test runners installed and a dummy test passes
Test plan doc	TC-001 to TC-012 written with expected outcomes
Bug template	GitHub Issues template live and usable
.env.example	All keys documented with example values

Watch out: If docker-compose is blocked on any machine, have that dev run postgres locally — but fix the compose setup before Day 2 starts.

DAY 2 — Authentication & Dashboards

Tuesday · Auth first

Login must work end-to-end before anything else is built on top of it.

Backend Team

- POST /api/v1/auth/login — validate credentials, bcrypt.compare, return JWT (1h) + refreshToken (7d)
- POST /api/v1/auth/register — validate all fields, hash password with bcrypt cost 12, send verification email via nodemailer
- GET /api/v1/auth/verify-email?token= — verify JWT token, set is_verified=true on users table
- POST /api/v1/auth/refresh — validate refresh token, return new token pair
- POST /api/v1/auth/logout — invalidate refresh token in DB (set to null)
- Write verifyJWT middleware and roleGuard(role) middleware; apply to all protected routes
- Test all auth endpoints in Postman; share collection with frontend team by EOD

Frontend Team

- Build Login page with React Hook Form + Zod validation; call POST /auth/login; store token in Zustand on success
- Redirect after login to role-specific dashboard (/student/dashboard, /teacher/dashboard, /admin/dashboard)
- Build Register page (student only) — all fields, inline validation on blur, show/hide password toggle
- Build Forgot Password and Reset Password pages (UI only on Day 2; wire up Day 3)
- Build all 3 dashboard skeletons: summary cards with hardcoded placeholder numbers, sidebar nav, page layout
- Use MSW (Mock Service Worker) to mock dashboard API responses while backend builds them

Testing Team

- TC-001: Login with valid credentials → expect 200 + JWT in response
- TC-002: Login with wrong password 5 times → expect 423 Locked after 5th attempt
- TC-007: Student calls POST /api/v1/assignments → expect 403 Forbidden
- TC-011: Expired JWT on any protected route → expect 401 Unauthorized
- Test register with duplicate email → expect 409 Conflict
- Test register with invalid email format → expect 422 with validation errors

DevOps Team

- Add EMAIL_USER, EMAIL_PASS, JWT_SECRET, REFRESH_SECRET to .env.example
- Set up Mailtrap account for dev email testing (free); share credentials with team
- Confirm GitHub Actions CI runs auth unit tests on PR to develop branch

End-of-day deliverables

Deliverable	Acceptance criteria
Login flow	JWT received → stored → redirect to correct dashboard
Register + verify	Email arrives in Mailtrap; account activates on click
Role guard	Student calling teacher API returns 403 (tested + passing)

Deliverable	Acceptance criteria
Postman collection	All auth endpoints shared with frontend team
Dashboard layouts	All 3 role dashboards render (with mock data)

Watch out: Frontend must NOT start wiring real dashboard API calls until backend confirms endpoints are tested and Postman collection is shared. Use MSW mocks in the meantime.

DAY 3 — Assignments & File Submissions

Wednesday · Core feature loop

The heart of SAIMS. Teacher creates → student submits → file lands in S3.

Backend Team

- GET /api/v1/assignments — paginated list scoped to user's courses; teacher sees all their own
- POST /api/v1/assignments — create assignment (draft or published); validate deadline is future date
- GET /api/v1/assignments/:id — full detail with rubric and attachment URL
- PUT /api/v1/assignments/:id — teacher can only update their own; 403 if not owner
- DELETE /api/v1/assignments/:id — only if zero submissions exist; return 409 if submissions exist
- POST /api/v1/files/upload — Multer streams to S3; validate MIME type server-side; return S3 URL
- POST /api/v1/submissions — check deadline, check attempt count, link file_url; set is_late flag
- GET /api/v1/assignments/:id/submissions — teacher only, paginated with status filter
- GET /api/v1/submissions/my?assignmentId= — student gets their own submission only

Frontend Team

- Build Assignment List page (student): fetch GET /assignments, deadline countdown chip (green/orange/red), submission status badge
- Build Assignment Submission page: file dropzone (react-dropzone) with upload progress bar, notes field, confirmation modal before submit
- Build Assignment Creation page (teacher): rich text editor (tiptap), dynamic rubric row builder (add/remove rows), deadline datetime picker
- Build Assignment Management page (teacher): submissions table, status filters (All/Submitted/Graded), click row to open grading
- Replace all MSW mock data with real API calls; handle loading and error states on every page

Testing Team

- TC-003: Submit assignment before deadline → 201 Created; check DB for submission record
- TC-004: Submit after deadline with allow_late=false → 403 Forbidden
- TC-010: Upload .exe file → 422 Invalid file type (test MIME check, not just extension)
- Verify uploaded file actually appears in S3 bucket at the returned URL
- Test student cannot access another student's submission via direct GET /submissions/:id (expect 403)
- Test teacher can list submissions for their assignment but not another teacher's

DevOps Team

- Add AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY, S3_BUCKET_NAME, AWS_REGION to .env.example
- Create S3 bucket with correct CORS policy (allow PUT from API server)
- Test file upload works end-to-end in dev environment; confirm 25MB limit enforced
- Cloudinary as fallback: if AWS setup is blocked, set up Cloudinary SDK (faster free tier setup)

End-of-day deliverables

Deliverable	Acceptance criteria
Assignment CRUD	Teacher creates assignment; appears in student list
File upload	File submitted → S3 URL stored in submissions table
Deadline gate	Late submission blocked (403) when allow_late=false
Postman updated	All submission + file endpoints added to collection
Real APIs wired	Zero MSW mocks remaining on Day 3 pages

Watch out: S3 credentials must be ready on Day 3 morning. If AWS IAM setup is slow, use Cloudinary — it has a simpler free tier and a 5-minute Node.js SDK setup.

DAY 4 — Grading, Internship & Notifications

Thursday · Close the feedback loop

Students must see their grade after the teacher reviews. This is the core value of SAIMS.

Backend Team

- PUT /api/v1/submissions/:id/grade — upsert grade record; store rubric scores as JSONB; if is_published=true, trigger notification
- POST /api/v1/notifications + GET /api/v1/notifications?userId= — insert and retrieve user notifications
- POST /api/v1/internships — admin creates internship record, assigns supervisor teacher, links student
- GET /api/v1/internships/my — student gets their active internship record
- POST /api/v1/internship-logs — validate one log per week (UNIQUE constraint); validate week not in future
- GET /api/v1/internship-logs?internshipId= — paginated log history for student or teacher
- PUT /api/v1/internship-logs/:id/review — teacher adds feedback text, sets status=reviewed
- Add audit log middleware: attach to all POST/PUT/DELETE routes; write actor_id, action, entity_type, entity_id, IP, timestamp

Frontend Team

- Build Review & Grading page: inline PDF viewer (react-pdf), rubric table with per-row marks input that auto-sums to total, feedback textarea (required before publish)
- "Publish Grade" button → confirmation modal → API call → notification sent → student's submission page immediately shows grade
- Build Internship Log Submission page: week form auto-calculates week number, multi-select skill tags, optional file upload
- Build Internship Review page (teacher): paginated log list, status chips (draft/submitted/reviewed), feedback form per log
- Build Notification bell: global header icon, unread count badge (red dot), click-through dropdown list
- Wire real dashboard API to all 3 dashboards (replace remaining mock data)

Testing Team

- TC-005: Grade submission with marks > assignment max_marks → 422 Unprocessable
- TC-006: Publish grade → is_published=true in DB + notification record created → student sees grade on their page
- Test internship log: submit the same week twice → system shows edit option, not a duplicate record
- Test teacher feedback text saves and appears in student's log history view
- Test audit_logs table: verify entry created for every PUT /grade and DELETE /users/:id call
- Performance: run k6 load test on GET /student/dashboard with 100 virtual users; assert p95 < 200ms

DevOps Team

- Confirm Redis is running in docker-compose and API connects successfully on startup
- Add Redis caching layer to GET /student/dashboard and GET /teacher/dashboard (5-minute TTL)
- Confirm cache hit vs miss in API logs; verify stale data cleared on grade publish
- Run initial k6 load test; document baseline p95 response times in README

End-of-day deliverables

Deliverable	Acceptance criteria
Full E2E lifecycle	Create → Submit → Grade → Notify → Student sees grade
Internship module	Log submission and teacher review both work
Notification bell	Unread count badge updates in real time after grade publish
Audit log	Every mutation has a corresponding audit_logs entry
Redis caching	Dashboard API returns cached response on second call

Watch out: Do NOT move to Day 5 until the grading → notification → student-sees-grade flow is tested end-to-end. This is the core value proposition of SAIMS.

DAY 5 — Admin Panel, Reports & Deployment

Friday · Ship it

Lock down the system. Admin manages everything. Staging deployment live by end of day.

Backend Team

- GET /api/v1/admin/users — paginated list with role, department, status filters
- POST /api/v1/admin/users — admin creates teacher or admin accounts (students self-register)
- PUT /api/v1/admin/users/:id — update any field; activate/deactivate (is_active toggle)
- DELETE /api/v1/admin/users/:id — soft delete (is_active=false); write audit log entry
- GET /api/v1/reports/assignments?from=&to=&courseId= — aggregate submission + grade summary
- GET /api/v1/reports/students?departmentId= — student performance aggregate
- GET /api/v1/reports/export?format=csv — stream CSV file download response
- Add express-rate-limit: 5 requests/15min on /auth endpoints; 100 req/min general API
- Add helmet.js (HTTP security headers) and strict CORS whitelist (no wildcard)
- Final security pass: confirm zero raw SQL string concatenation; all inputs via express-validator

Frontend Team

- Build Admin User Management page: data table with search, role filter, activate/deactivate toggle, create user modal
- Build Reports page: filter bar (date range, department, course), preview table, "Export CSV" download button
- Build Profile page for all roles: update name, email, change password form
- Final UI pass on all pages: empty states on every list, error boundaries, loading spinners on every async action
- Accessibility pass: keyboard navigation on all forms, ARIA labels on all icon buttons, visible focus rings
- Remove all remaining console.log statements; verify no 404s in Network tab on any page

Testing Team

- TC-008: Admin deletes user → is_active=false in DB + audit_logs entry created
- TC-009: Register with duplicate student_id → 409 Conflict
- TC-012 (Playwright E2E): Register → verify email → login → submit assignment → teacher grades → student sees grade
- E2E teacher lifecycle: login → create assignment → view submissions → grade → publish
- Security: student calls GET /admin/users → 403; student calls PUT /submissions/:otherId/grade → 403
- Final smoke test: open all 10 pages, check for console errors, verify all API calls succeed

DevOps Team

- Write production Dockerfile for backend: multi-stage build (builder → runner); final image < 200MB
- Write production Dockerfile for frontend: build React app → serve with Nginx alpine
- Push Docker images to GitHub Container Registry (ghcr.io)
- Deploy to staging on Render.com (free tier) or Railway — set all env variables in dashboard
- Run DB migrations on staging database (npm run migrate)
- Smoke test staging URL: login as each role, submit an assignment, publish a grade — all must work
- Set up UptimeRobot free monitoring on staging URL; configure email alert on downtime
- Update README: local setup instructions, staging URL, environment variable reference

End-of-day deliverables

Deliverable	Acceptance criteria
Admin panel	User CRUD works; deactivated users cannot login
CSV export	Report downloads as valid CSV file
Staging URL	Live and accessible by all team members
E2E tests	Full student + teacher Playwright workflows passing
Security	Rate limiting, helmet, CORS all active on staging
README	Any new dev can set up locally in under 10 minutes

> **Watch out:** Deploy to staging even if some features are incomplete. A broken feature on a live staging URL is better than a perfect feature that only runs on one laptop.

Daily Coordination Rules

Git workflow

- All work on feature/* branches — never commit directly to develop or main
- Every PR requires 1 reviewer approval before merge
- PR description must reference the GitHub board card number
- Merge to develop triggers CI; merge to main triggers staging deploy

Communication rhythm

Meeting	Time	Duration	Who	Purpose
Morning kickoff	9:00 AM	15 min	All teams	Review board, unblock issues, confirm who has what
Backend → FE handoff	As needed	10 min	BE + FE leads	Share new Postman endpoints; clarify request/response
EOD check-in	5:30 PM	15 min	All teams	Demo what was built, confirm deliverables met, flag blockers
Async standups	Any time	—	Individual	Post in Slack/Discord: Done / Doing / Blocked

API contract rule

The backend team defines the API contract (endpoint, method, request body, response shape) BEFORE writing the implementation. This goes into the shared Postman collection. The frontend team can build against mock data using MSW until the real endpoint is ready — nobody waits.

Definition of done (per task)

- Code is committed to a feature branch and pushed to GitHub
- All unit/integration tests for the feature are passing
- The corresponding GitHub board card is moved to "Review"
- For backend tasks: endpoint is added to Postman collection with example request/response
- For frontend tasks: no console errors on the page; loading and error states handled
- PR is opened with a description — not just a commit title